All-in-One Revision Notes: Mobile Programming

Here are comprehensive revision notes covering the outlined plan, drawing information from the provided sources and including additional insights where noted.

***

1. Introduction to Mobile Programming

* **Mobile (as a concept):** Refers to being "able to move freely or easily".

* **Mobile Device:** A portable, handheld communications device connected to a wireless network, enabling voice calls, text messages, and application execution on the go.

* **Mobile Network (Cellular Network):** A communication network where the final link is wireless, distributed over land areas called cells, each served by a base station. This base station provides network coverage for voice, data, and other transmissions. The Mobile Switching Station Center (MSC) controls network switching in 2G core networks.

* **Mobile Station (MS):** Comprises all user equipment and software required for communication with a mobile network, such as a mobile phone with calling and SMS applications installed.

* **Mobile Service:** A provision delivered through mobile devices. Examples include:

    * **Short Messaging Service (SMS):** Provision of text messages (usually limited to 160 characters) to mobile phones from the internet or other mobile devices. SMS provides a store-and-forward system and low-cost non-voice communication.

    * **Multimedia Message Service (MMS):** A media-rich extension of SMS allowing pictures, sounds, or low-quality videos over a wireless network.

    * **Unstructured Supplementary Service Data (USSD):** User-initiated services where a code is entered on the device and sent as a request to the network, creating a real-time, two-way interaction session (e.g., *144# to check airtime balance).

* **Application (App):** Software developed specifically for smartphones and other mobile devices.

Mobile Application Development:

This refers to the act or process of developing application software for mobile devices. Before development, several factors should be considered:

* **Choosing the Mobile Platform:** This depends on factors like target users (e.g., BlackBerry for enterprise, Android/iOS for mass market) and unique platform features.

* **Familiarizing with the Application Framework:** These are sets of class libraries that provide an interface for developers to build applications.

* **Knowing Device Specifications:** Understanding screen dimensions, button positions, processing power, graphics capabilities, and multi-touch capabilities provides a better user experience.

* **Exploring Available Tools:** Determining the best tools for development, such as emulators for testing and Integrated Development Environments (IDEs).

Integrated Development Environments (IDEs):

IDEs are sets of development tools with a window interface for editing, compiling, running, or viewing mobile application output. They also track projects and help generate UI code.

* **Categories:**

  * **Open IDEs:** Freely available, e.g., Eclipse and Android Studio for Android.

  * **Proprietary IDEs:** Require subscription fees or licenses, e.g., Xcode for iOS and MS Visual Studio for Windows Phone.

* **Components of an IDE:** Program Editor, Compiler, Testing tools (Emulator test management, unit tests), Debugging tools.

* **Android Studio:** The official IDE for Android app development, based on IntelliJ IDEA. It includes features like a flexible Gradle-based build system, a fast emulator, Instant Run, code templates, GitHub integration, extensive testing tools, Lint tools, C++ and NDK support, and Google Cloud Platform integration.

Motivations for using Mobile Devices:

* **Ubiquity:** Potentially available everywhere and any time due to portability.

* **Integration into Daily Life:** Used as a utility for various activities like work, play, and study.

* **User Representation:** Can represent the user, e.g., through a phone number.

* **Interactive Applications:** Enables communication through feedback channels like social media.

* **Multi-modal Content:** Allows access to video and audio clips.

* **Dynamic Content Update:** Content within the same file can be updated dynamically via features like copy and paste.

Mobile Application Development Challenges:

* **No "Standard" Device:** Lack of a single device for all platforms (iOS, Windows Phone, etc.).

* **Low Bandwidth Input:** Often limited input capabilities.

* **Limited Screen Size:** Small screen real estate.

* **Unreliability in Connectivity:** Inconsistent network access.

* **Small Keyboard:** Less convenient for extensive typing.

2. Types and Examples of Mobile Applications

Mobile applications are primarily categorized into three types: **Native**, **Web**, and **Hybrid**.

* **Native Applications:**

  * **Definition:** Built specifically for each mobile platform and installed directly on the device. A separate version is required for each platform if cross-platform functionality is desired.

  * **Development Languages:** Object C for iOS, **Java for Android** and BlackBerry RIM, C# for Windows Phone.

  * **Advantages:**

    * **Full Hardware/Software Access:** Can access on-board hardware (GPS, camera, microphone, graphics) and software (email, calendar, contacts, gallery, file manager, home screen widgets).

    * **Offline Functionality:** Remains installed, requiring no internet connection to run after initial download.

    * **Performance and Security:** Generally offer better performance and security than web apps.

    * **Monetization:** Easier to sell through app stores.

  * **Disadvantages:**

    * **Platform Specificity:** Cannot work across different mobile platforms without separate versions.

    * **Higher Cost/Effort:** Requires more money and effort for development, testing, and distribution of multiple versions.

*   **Examples:**

    *   **SMS Applications:** Used for transferring short text messages. Examples include Chomp SMS, EvolveSMS, and Handcent.

    *   **USSD Applications:** Menu-based apps using USSD protocols to create real-time connections with service providers. The "Default Phone app" (dialer) is an example, often used for checking balances (*144#). M-Pesa in Kenya is a successful mobile money-transfer application that utilizes USSD and SMS.

    *   **Instant Messaging (IM) Applications:** Enable real-time message exchange over the internet for smartphone users. Examples include BlackBerry Messenger, WeChat, WhatsApp, Google Talk, and Skype.

    *   **Quick Response (QR) Code Applications:** Apps that read scannable barcodes (QR codes) using the camera to provide information like URLs, contact details, or trigger downloads.

*   **Why Native Apps for Complex Student Information (KCA University Scenario):** A native app would be the best choice for a student information management application with "many diverse aspects" and "potential to do many things" because native apps can access the full range of on-board hardware and software, offer better performance for complex graphics and operations, and generally provide higher security and better integration with system functionalities. This allows for a richer user experience and robust functionality.

*   **Web Applications (Mobile Website):**

    *   **Definition:** A mobile website formatted for mobile devices, accessed through the device's web browser. Typically downloaded from a central web server each time they run, though HTML5 apps can run offline.

    *   **Technologies:** Developed using HTML (for static text/images), Cascading Style Sheets (CSS) (for style/presentation), and JavaScript (for interactions/animations).

    *   **Advantages:**

        *   **Cross-Platform Compatibility:** Can reach the broadest audience with less effort due to standardized mobile web browsers.

        *   **Cost/Time Efficient:** Relatively cheap, easy, and fast to build.

        *   **Integration with Existing Systems:** Can integrate with existing systems and databases.

    *   **Disadvantages:**

        *   **Limited Hardware/Software Access:** Generally cannot access on-board hardware/software like cameras or direct GPS control, nor support heavy graphics.

        *   **Network Requirement:** Typically require a network connection to function, with performance issues if the website is slow.

    * **Security:** While mentioned as better for security in one source, another source implies native apps generally offer better security due to deeper system integration and sandboxing. (This is a point of potential contradiction/nuance in the sources, and depends on the specific security aspects being considered. For example, web apps are less susceptible to device-specific malware, but native apps can implement more robust client-side security).

* **Tools:** Wapple, mobiSiteGalore, MobilePress (WordPress plugin), MobileFrame, Pyxis.

* **Hybrid Applications:**

* **Definition:** Combines elements of both native and web applications, where the UI appears in a browser window wrapped within a native app.

* **Advantages:**

    * **Device Functionality Access:** Can access device functionality (onboard software and hardware) not available through a browser.

    * **Reduced Development Time/Cost:** Allows for code reuse across different devices with minimal customization.

    * **App Store Distribution:** Can be downloaded from app stores and stored/launched like native apps.

* **Disadvantages:**

    * Requires some customization (writing a few codes) to reuse across different devices.

* **Tools:** PhoneGap/Cordova, Intel XDK.

    * **Additional Tools (External Information):** Other popular frameworks for hybrid/cross-platform development include **Ionic**, **React Native**, and **Flutter**. These are not mentioned in the provided sources.

Comparison of Application Types:

* **Native vs. Web:** Native apps are installed directly on each device and are platform-specific, while web apps are served from a central location, accessed via a browser, and are platform-independent.

* **Hybrid vs. Web:** Both can work across different platforms. However, hybrid apps can access the device's on-board hardware and software, which web apps generally cannot.

* **Native vs. Hybrid:** To the user, a well-designed hybrid app can appear very similar to a native app. The key difference for developers is that hybrid apps allow for significant code reuse (HTML, CSS, JavaScript) across platforms, whereas native apps require separate rewrites for each platform.

3. Android Stack (Architecture Layers)

The Android system is a **full software stack** typically divided into four areas or layers. Google leads the Android Open Source Project (AOSP) responsible for its development.

* **1. Applications:**
    * This is the highest layer.
    * The AOSP includes several default applications such as Browser, Camera, Gallery, Music, and Phone.
    * Widgets are also part of this layer, operating in small areas of the Home screen.

* **2. Application Framework:**
    * An API that allows high-level interactions with the Android system from Android applications.
    * It encapsulates the Linux kernel, libraries, and runtime.
    * Android application developers typically work with this layer (and the Applications layer) to create new apps.
    * **Components/Important Classes:**
        * **Activity Manager:** Manages the lifecycle of applications and provides a common navigation back stack.
        * **Package Manager:** Keeps track of installed applications.
        * **Window Manager:** Manages windows.
        * **Telephony Manager:** Contains APIs for building phone applications.
        * **Content Providers:** Allow applications to share data with other applications (e.g., contact info).
        * **Resource Manager:** Used to store localized strings, bitmaps, layout file descriptions, etc..
        * **View System:** Contains all building blocks of the UI, used to create input controls like buttons, menus, and text fields.
        * **Location Manager:** Provides support for location-based services.
        * **Notification Manager:** Handles user notifications about events, enabling applications to display alerts in the status bar.

* **Extensible Messaging and Presence Protocol (XMPP):** An open XML technology for real-time communication (instant messaging, shared editing), allowing applications to transmit messages between devices. It works with any Gmail account and can deliver server-to-server messages.

* **3. Libraries and Runtime:**

* This layer includes libraries for many common functions of the Application Framework (e.g., graphic rendering, data storage, web browsing).

* It also contains the **Dalvik runtime** (now generally superseded by ART for modern Android, though the source mentions Dalvik) and core Java libraries for running Android applications.

* **Native Libraries:** Shared libraries written in C or C++, compiled for the specific hardware architecture. Important libraries include:

    * **OpenGL-ES (OpenGL):** Supports 2-D and 3-D graphics.

    * **SQLite database:** An embedded database for data storage.

    * **WebKit library:** For browsing HTML content.

* **4. Linux Kernel:**

* The Android operating system is **based on the Linux kernel**.

* This is the communication layer for the underlying hardware.

* **Major functionalities:** Low-level memory management, process management, networking, and other OS-related services. It also acts as a hardware abstraction layer (HAL).

Advantages of the Android Platform:

* **Open:** Developers can access handset features or the same application framework as core phone applications.

* **Free:** Development tools are freely available for download, with only a registration charge for initial application publishing (subsequent updates are free).

4. Application Life Cycle (Activity Life Cycle)

* **Activity:** Internally, each user interface screen in an Android application is represented by an `Activity` class. An activity presents a visual user interface for one task. Users can move through screens by starting other activities.

* **Activity Life Cycle:** The steps an activity goes through from its start state to its exit state.

* **Activity vs. Process Life Cycle:** An application consists of one or more activities plus a Linux process to contain them. The activity life cycle is **not tied to the process life cycle**; processes are disposable containers for activities, meaning an application can be "alive" even if its process has been killed.

Activity States: During its lifetime, an Android activity can be in one of the following states:

1. **New Activity State:** A new activity being loaded into memory (allocated memory).

2. **Active or Running State:** In the foreground of the screen (at the top of the activity stack for the current task). This is the activity receiving user input.

3. **Paused Activity:** Has lost focus but is still visible to the user (another activity is on top of it, but is transparent or doesn't cover the full screen). A paused activity is completely alive, maintains all state, but can be killed by the system in extreme low memory situations.

4. **Stopped Activity:** Completely obscured by another activity. It retains all state and member information but is no longer visible. Its window is hidden, and it is often killed by the system when memory is needed.

5. **Destroyed Activity:** Killed (dropped) to reclaim memory. An activity no longer in the activity stack.

* **Killable States:** Paused state and Stopped state are "killable states," meaning they can be followed by activity termination (destroyed state).

Activity Lifecycle Transition Methods:

There are seven transition methods that define an activity's entire lifecycle. These methods need to be overridden in the activity class for Android to call them at the appropriate time.

1. **`onCreate()` Method:**

    * The **first method called** when an activity is first created (starts up).

    * Used for initial setup of "global" state and one-time initialization, such as creating the user interface (views, binding data to lists).

    * Receives a `Bundle` object containing the activity's previous state, if captured.

* Always followed by `onStart()`.

2. **`onStart()` Method:**

* Called just before the activity becomes **visible** to the user.

* It is followed by `onResume()` if the activity comes to the foreground, or `onStop()` if it becomes hidden.

* Marks the beginning of the **Visible Lifetime** (between `onStart()` and `onStop()`), during which the user can see the activity.

3. **`onResume()` Method:**

* Called when an activity result or a new intent is delivered.

* At this point, the activity is **at the top of the activity stack** and interacting with the user.

* A good place to start animations and music.

* Always followed by `onPause()`.

* Marks the beginning of the **Foreground Lifetime** (between `onResume()` and `onPause()`), during which the activity is in front of all others and interacting with the user. An activity can frequently transition between resumed and paused states.

4. **`onPause()` Method:**

* Called when the device goes to sleep or a new activity is started.

* Runs when the activity is about to go into the background, usually because another activity has been launched in front of it.

* Typically used to commit unsaved changes to persistent data, stop animations, and release resources consuming CPU.

* Followed by `onResume()` (if it returns to the front) or `onStop()` (if it becomes invisible).

* The activity in this state is **killable** (destroyable) by the system.

5. **`onStop()` Method:**

* Called when the activity is **no longer visible** to the user and won't be needed for a while (e.g., being destroyed or covered by another activity).

* Followed by `onRestart()` (if coming back to user interaction) or `onDestroy()` (if going away).

* The activity in this state is **killable** by the system. If memory is tight, `onStop()` may not be called, and the process might simply terminate.

6. **`onRestart()` Method:**

* Called after the activity has been stopped, just before being started again.

* Indicates that the activity is being redisplayed to the user from a stopped state.

* Always followed by `onStart()`.

7. **`onDestroy()` Method:**

* The **final call** an activity receives, just before it is destroyed (killed).

* Can be called because the activity is finishing or because the system is temporarily destroying the instance to save space.

* Used to release all remaining resources.

* The activity in this state is **killable** by the system. If memory is tight, `onDestroy()` may not be called.

Other Activity Methods:

* **`onSaveInstanceState(Bundle)`:** Called to allow the activity to save per-instance state (e.g., cursor position in a text field). The default implementation automatically saves state for UI controls, so overriding is often unnecessary.

* **`onRestoreInstanceState(Bundle)`:** Called when the activity is being reinitialized from a previously saved state. The default implementation restores UI state.

Activity Transitions Diagram (from Joyce's scenario):

* **Scenario Description:** Joyce is listening to music (Music App: `Running`). A call comes in (`Phone App: New/Active`). She answers (`Phone App: Active`, Music App: `Paused`). She then takes a selfie and forwards it via WhatsApp (`Camera App: New/Active`, `WhatsApp App: New/Active`). After sending, she closes all other apps except music.

* **Diagram Explanation (Conceptual, based on CAT II principles):**

  * **Music App (Activity A):**

    * `onCreate()` -> `onStart()` -> `onResume()` (Running).

    * When call comes in/WhatsApp is opened: `onPause()` (Music App is still visible behind) -> `onStop()` (Music App is completely obscured if other apps take full screen).

    * When other apps are closed and music continues: `onRestart()` -> `onStart()` -> `onResume()` (Music App returns to foreground).

* **Phone App (Activity B):**

    * `onCreate()` -> `onStart()` -> `onResume()` (when call is active).

    * `onPause()` -> `onStop()` (when WhatsApp/Camera takes over, or if call is put on hold and not the primary focus).

    * `onDestroy()` (when call ends and app is closed).

* **Camera App (Activity C) / WhatsApp App (Activity D):**

    * `onCreate()` -> `onStart()` -> `onResume()` (when launched).

    * `onPause()` -> `onStop()` -> `onDestroy()` (when user finishes tasks and closes them, possibly implicitly by system if memory is needed).

5. Application Building Blocks (Components of Android Application)

An Android application is composed of several fundamental components, also known as building blocks or application framework elements.

* **Activities:**

    * As discussed, an activity is a class that presents a **visual user interface (screen)** for a single task the user can perform.

    * Activities are organized in an **activity stack**, where new activities are placed on top, and only one is visible at a time.

* **Intents:**

    * An intent is a **message** that describes a desired action, the data to be acted upon, and the category of component that should perform the action.

    * They are primarily used to **start or activate** the three main application components: Activities, Services, and Broadcast Receivers.

    * An intent can be received by zero or more components.

    * **Primary Attributes:**

    * **Action:** The general action to be performed (e.g., `ACTION_VIEW`, `ACTION_DIAL`).

    * **Data:** The URI of the data to be acted upon (e.g., a phone number for `ACTION_DIAL`).

* **Types of Intents:**

    * **Directed (Explicit) Intent:** Has one specific recipient. For example, Activity A starting Activity B.

    * **Broadcast (Implicit) Intent:** The message can be received by anyone interested. Broadcasted by one component, and filtered by intent filters.

* **Invoking Intents:** Intents are invoked using methods like `startActivity()` for activities.

    * *Example for launching activity:* `Intent intent_object = new Intent(this_activity, target_activity_class); startActivity(intent_object);`.

    * *Example for dialing a number:* `Intent i = new Intent(android.content.Intent.ACTION_DIAL, Uri.parse("tel:+254722xxxxxx")); startActivity(i);`.

* **Secondary Attributes:**

    * **Category:** Provides additional information about the components that handle the intent.

    * **Type:** Describes the MIME type of the intent data. If not specified, Android infers the type.

    * **Component:** Explicitly names the component (class) that should receive the intent (used for explicit intents).

    * **Extras:** Additional information associated with the intent, treated as key-value pairs (`Bundle`). Example: `intent.putExtra("key", "value");`.

    * **Flag:** Describes how the intent should be handled (e.g., `FLAG_ACTIVITY_NEW_TASK` to start a new task).

* **Services:**

    * A service is a class without a **visual interface** that executes in the **background**.

    * Services do not interact with the user directly and are typically used for performing **long-running tasks**.

    * Examples include a music player that keeps playing while the user navigates elsewhere, or a searching service fetching data over the network without blocking user interaction.

* **Content Providers:**

* Objects that can **retrieve and store data** in data storage mechanisms (e.g., SQLite or data files).

* They are the **only way to share data across packages/applications** (e.g., any application can access the contacts database).

* **AndroidManifest.xml:**

* A crucial file that is **automatically generated** for every Android application.

* **Functions:**

* Describes application components (activities, services, broadcast receivers, content providers).

* Declares the application's Java package name.

* Declares the **minimum Android SDK version** required to run the application (`minSdkVersion`).

* Declares **permissions** the application must have to access certain APIs (e.g., Camera, Bluetooth) or other applications.

* Declares permissions that others need to access its own components.

* Defines **intent filters**, informing the system which implicit intents the component can handle (e.g., `ACTION_MAIN`, `CATEGORY_LAUNCHER` for the main activity).

* **Broadcast Receivers:**

* A class that receives and **reacts to broadcast intents**.

* Implemented as a subclass of `BroadcastReceiver`.

* Examples of broadcast intents include low battery, power connected, device shutdown, or timezone changes.

* Applications can initiate broadcasts using methods like `Context.sendBroadcast()`.

* **Resources:**

* External files and static content used by an application, such as strings, drawables, layouts, and constants.

* **Layouts:**

* Refer to **ViewGroups** that control the format and appearance of the views (UI elements).

* Examples include Linear layout, Relative layout, Table layout, Grid layout, and Tab layout.

6. Introduction to XML

* **Programming Methodologies for Mobile Applications:**

* **Declarative Design Method:** Involves writing code that specifies *what* should be done by the mobile device. In Android, **eXtensible Markup Language (XML) is used** for declarative programming to describe what will be viewed by the user.

* **Procedural Design Method:** Involves writing code that specifies *how* user interface objects will be created and manipulated. In Android, **Java programming language is used** for procedural design.

* **Choice of Design Method:** Android allows both, but **XML is recommended** because it requires fewer lines of code and is generally easier to understand than the corresponding Java code for UI design.

* **Purpose of XML in Android:** Describes the UI components (e.g., push buttons, labels, text fields) and defines information to be stored or transferred. Data in external XML files can be available to various mobile devices.

The Difference Between XML and HTML:

* **Purpose:** XML was created to **define, store, and transfer information**, focusing on the data itself. HTML was designed to **display data**, focusing on how the data looks.

* **Tags:** HTML documents use predefined tags (e.g., `<h1>`, `<p>`). XML allows authors to **define their own tags** and document structure.

Terminologies Used in XML:

1. **XML Tag:** A construct for describing elements of a document.

   * **Start-tags:** Begin an element (e.g., `<TextView>`).

   * **End-tags:** End an element (e.g., `</TextView>`).

   * **Empty-element tags:** Do not have end-tags and must be self-terminated with a `/` (e.g., `<TextView />`).

   * **Custom Tags:** XML tags are not predefined; you must define your own.

* **Properly Nested:** Tags must be nested correctly (e.g., `<LinearLayout><TextView>...</TextView></LinearLayout>` is correct, `<name><email>...</name></email>` is not).

2. **XML Element:** A logical document component starting with a start-tag and ending with a matching end-tag, or consisting of an empty-element tag. Its content can include other elements (child elements), text, or attributes.

* **Example:** In `<resources><color name="opaque_red">#f00</color></resources>`, `<resources>` contains `<color>`, and `<color>` contains text content and an attribute.

3. **XML Attributes:** Provide information that is not part of the primary data, expressed as name/value pairs within an element's tag.

* **Example:** `<file type="gif">computer.gif</file>` where `type="gif"` is an attribute.

* **ID References:** Assigned to elements for identification (e.g., `android:id="@+id/btnlogin"`).

4. **XML Namespaces (`xmlns`):** A mechanism to avoid element name conflicts by assigning unique names to groups of elements/attributes. Identified by a Uniform Resource Identifier (URI).

* **Declaration:** `<element xmlns:name="URL">`.

* **Example in Android:** `xmlns:android="http://schemas.android.com/apk/res/android"`. This associates elements and attributes with the `android:` prefix to that specific namespace. When a namespace is defined for an element, all child elements with the same prefix are associated with it.

* **Uniform Resource Identifier (URI):** A string of characters identifying an Internet Resource, enabling interaction over a network.

    * **Uniform Resource Locator (URL):** Identifies an Internet domain address, specifying resource location and retrieval protocol (e.g., `http://`).

    * **Uniform Resource Name (URN):** A URI that defines the identity of a resource but does not imply its availability (e.g., `urn:isbn:0451450523`).

XML Document Tree Structure:

* XML documents must contain a **single root element**, which is the "parent" of all other elements.

* The tree starts at the root and branches down.

* Elements can have **sub-elements (child elements)**.

* **Parent, Child, Sibling:** Terms used to describe relationships between elements (e.g., in `<bookstore><book>...</book></bookstore>`, `<bookstore>` is the parent of `<book>`, and multiple `<book>` elements would be siblings).

XML Syntax Rules:

1. **Closing Tags:** It is illegal to omit the closing tag in XML; all elements must have one (e.g., `<EditText>...</EditText>`). Self-closing tags use `/>`.

2. **Case Sensitivity:** XML tags are case-sensitive. `<TextView>` is different from `<textview>`. Opening and closing tags must match in case.

3. **Root Element:** XML documents must have exactly one root element.

4. **Attribute Values Quoted:** All attribute values in XML must always be quoted (e.g., `android:layout_width="fill_parent"`).

5. **Comments:** Used to explain code and are ignored by the CPU. Syntax: `<!-- This is a comment -->`.

6. **Whitespace Preservation:** Whitespace within a document is preserved, not truncated (e.g., "**User Name**" will appear as "**User Name**" if defined with multiple spaces).

XML Naming Rules:

1. Names can contain letters, numbers, and other characters.
2. Names cannot start with a number or punctuation character.
3. Names cannot start with the letters `xml` (or `XML`, `Xml`, etc.).
4. Names cannot contain spaces.
5. Any name can be used; no words are reserved.
6. Names should be meaningful.

7. Input Controls and User Interface Layouts

User Interface Terminologies:

* **Orientation:** The screen's orientation from the user's view. **Landscape** has a wide aspect ratio, while **Portrait** has a tall aspect ratio. Orientation can change at runtime when the user rotates the device.

* **Resolution:** The total number of physical pixels on a screen.

* **Screen Size:** Actual physical size, measured diagonally. Android categorizes all actual screen sizes into four generalized sizes: **small, normal, large, and extra large**.

* **Screen Density:** The quantity of pixels within a physical area, typically measured in **dpi (dots per inch)**. Android categorizes densities as **low, medium, high, and extra high**. A "low" density screen has fewer pixels in a given area compared to "**normal**" or "**high**".

* **Density-independent pixel (dp):** A virtual pixel unit used for defining UI layout to ensure density-independent sizing. 1 dp equals 1 physical pixel on a 160 dpi (medium density) screen. The system scales dp units transparently based on actual screen density (e.g., on a 240 dpi screen, 1 dp = 1.5 physical pixels). This ensures proper UI display across different screen densities.

  * Generalized dp sizes: xlarge (>= 960dp x 720dp), large (>= 640dp x 480dp), normal (>= 470dp x 320dp), small (>= 426dp x 320dp).

User Interface Layout:

The UI layout of an Android application is defined using a **hierarchy of Views and ViewGroups**. In a single UI, they are organized into a hierarchical tree structure, with ViewGroups at the root and intermediate positions, and Views at leaf nodes. A UI can only have one root element.

* **View (Widgets):**

  * A class that draws **user interface (UI) objects** on the screen that the user can interact with.

  * Often referred to as **widgets**.

  * **Examples:** `TextFields`, `EditFields`, `Buttons`, `Checkboxes`, `RadioButtons`.

  * **Common Operations:**

    * **Set properties:** Using XML attributes (e.g., `layout_width="fill_parent"`).

    * **Set focus:** Using `requestFocus()` method in response to user input.

    * **Set up listeners:** To be notified when events occur (e.g., a button click).

    * **Set visibility:** Using `setVisibility(int)` to hide or show views.

* **ViewGroup (Layouts):**

  * A class that acts as a **container for holding views**, including other ViewGroups, to organize the layout of the interface.

  * **Layouts** are a type of ViewGroup.

  * Their main purpose is to hold and organize visual elements (controls) like text controls, buttons, or images on the screen.

* Layout resources are stored as XML files in the `/res/layout` resource directory.

* The XML structure for a UI typically follows: `<viewgroup><view>...</view><viewgroup><view>...</view></viewgroup></viewgroup>`.

Common Layouts Provided by Android:

1. **LinearLayout:**

* Arranges controls (its children) in a **single column (vertical orientation) or single row (horizontal orientation)**.

* In a vertical list, there's only one child per row, regardless of width. In a horizontal list, there's only one row high.

* Commonly used.

* **Attributes:** `android:orientation="vertical"` or `"horizontal"`, `android:layout_width="fill_parent"` (or `match_parent` for API Level 8+) and `android:layout_height="fill_parent"` (or `match_parent`) to make the layout proportional to the screen.

2. **RelativeLayout:**

* Arranges view objects (children) in **relation to each other or to the parent**.

* Often used to create forms.

* Children are identified by unique IDs (e.g., `android:id="@+id/ok"`), where `+id/` creates a new resource ID in `R.java`.

* **Attributes:** `android:layout_alignParentRight="true"` (aligns to parent's right), `android:layout_toLeftOf="@id/ok"` (aligns left of another control by ID), `android:layout_marginLeft="10dp"` (aligns 10dp from left margin).

3. **TableLayout:**

* Arranges controls in **rows and columns**, similar to an HTML table.

* Each row (`<TableRow>` element) has zero or more cells, defined by any kind of `View`.

* **Attributes:** `android:padding="3dip"` (sets space inside border, between border and content), `android:gravity="right"` (positions contents within the view), `android:layout_gravity="right"` (positions the view relative to its parent).

4. **FrameLayout:**

* Creates a **blank space** on the screen that can be filled with a **single object** (e.g., a picture to swap in/out).

* All children start at the top-left of the screen, and subsequent children are drawn over previous ones, potentially obscuring them.

* Generally used to hold a single child view due to difficulty organizing multiple views scalably without overlap.

* Mostly used for **tabbed views and image switchers**.

* **Attributes:** `android:src` (assigns an image), `android:text` (assigns text), `android:scaleType="fitCenter"` (adjusts object to fit center of container proportionally).

5. **Tab Layout:**

* Has two main elements: a **tab indicator** (e.g., Label or Label and Icon) and **content** (e.g., View or Activity).

Loading Layout XML Resource:

Android compiles XML layout code, which is then loaded in Java code using the `setContentView()` method, typically within the `onCreate()` method of an activity (e.g., `setContentView(R.layout.<layout_file_name>);`).

Padding vs. Margins:

* **Padding:** The space *inside* the border, between the border and the actual view's content.

* **Margins:** The spaces ***outside*** the border, between the border and other elements next to the view.

Enabling Interactivity (Procedural Design Method in Java):

To enable interactivity (listen and respond to user events), developers perform the following tasks:

1. **Declare Reference Variables:** Declare variables to hold references of `Input` controls defined in the XML layout file (e.g., `ViewType Identifier;`).

2. **Initialize Reference Variables:** Initialize these variables with the IDs of the `Input` controls using the `findViewById()` method within the `onCreate()` method (e.g., `Reference variable = findViewById(R.id.idname);`).

3. **Event Handling:** The activity of capturing events (e.g., button presses, screen touch) and taking appropriate action. Android maintains an event queue as FIFO.

* **Event Handler:** The method that actually handles a registered event.

*   **Event Listener:** An interface in the `View` class that contains a method for calling the event handler when an event occurs.

*   **Event Listener Registration:** Linking an event listener to a user input control (e.g., a button) so the event handler is called upon user action. *Example:* `button.setOnClickListener(this);` (if the activity implements `OnClickListener`).

8. Application Resources

*   **Definition:** Resources are additional files and static content used by an application.

*   **Types of Resources:** Color state list resource, Drawable resources, Layout resource, String resources, Menu Resource, etc..

*   **Android Project Directories and Files (Non-Code Resources):**

    *   `res/` (resources): Contains all non-code resources.

        *   `res/drawable/`: Stores drawable resources like images.

        *   `res/drawable-hdpi/`, `res/drawable-mdpi/`, `res/drawable-ldpi/`: Density-specific folders for images (high, medium, low dpi).

        *   `res/layout/`: Stores layout XML files.

        *   `res/values/`: Stores string and color resources (and other simple resources).

    *   `AndroidManifest.xml`: Describes application components, package name, SDK version, and permissions.

    *   `java/`: Contains Java source code files.

    *   `gen/`: Contains Java files generated by Android Studio, such as `R.java` (which links resources to Java code).

*   **String Resources:**

    *   **Purpose:** Provides text strings for the application with optional text styling and formatting.

    *   **Storage:** Saved in `res/values/strings.xml`.

    *   **Access:** Referenced from `R.string` (or `R.array`, `R.plurals`) classes.

    *   **Structure:** Defined within a `<resources>` root element, using `<string>` tags with a `name` attribute to provide a unique ID (e.g., `<string name="hello">Hello!</string>`).

    *   **Application:** Applied to UI elements via the `android:text` attribute in XML layout files (e.g., `android:text="@string/hello"`).

* **Color Resources:**

  * **Purpose:** Defines color values.

  * **Storage:** Created in `res/values/` (e.g., `colors.xml`).

  * **Structure:** Within a `<resources>` root, use `<color>` tags with a `name` attribute and a hexadecimal color value (e.g., `<color name="opaque_red">#f00</color>`).

  * **Application:** Used anywhere a hexadecimal color value is accepted, such as `android:textColor` or `android:background` attributes in XML layout files.

* **Drawable Resources:**

  * **Definition:** A general abstraction for "something that can be drawn." Unlike a `View`, a `Drawable` does not receive events or interact with the user.

  * **Saving:** Saved in `res/drawable/` or density-specific subfolders (`res/drawable-hdpi`, etc.). The application accesses them based on the device's screen density.

  * **Types of Drawables:**

    * **Bitmap Graphic:** (.png, .jpg, or .gif).

    * **Nine-Patch Drawables:** An extension to the PNG format that specifies how to stretch and place content inside (PNG with extra pixels for stretchable regions and content bounds).

    * **Shape:** Defines a shape (Rect, Oval, Line, Ring) and its fill/stroke/color. Uses simple drawing commands, allowing better resizing than raw bitmaps.

    * **State List:** A compound drawable that specifies a set of drawables for different view states (e.g., pressed, enabled, selected) and selects one based on its current state. Mostly used for buttons.

    * **Level List:** A compound drawable that selects one of a set of drawables based on its level (e.g., a progress bar changing based on WiFi signal strength). `LevelListDrawable.setLevel()` changes the image.

    * **Layer List:** A compound drawable that draws multiple underlying drawables on top of each other (the last one is on top). Can add simple effects to existing PNG drawables.

    * **Scale Drawables:** A compound drawable that changes the size of another Drawable (child drawable) based on its current level value, often used for progress bars.

  * **`ImageView` Element:** Used to display an image.

    * **Attributes:**

      * `android:id="@+id/image_display"`: Sets the ID for referencing the widget in Java code.

* `android:src="@drawable/welcome"`: References a drawable resource named `welcome` from the `res/drawable` folders (no extension needed).

* `android:layout_gravity="center_horizontal|center_vertical"`: Positions the `ImageView` at the center both horizontally and vertically.

* `android:layout_width="wrap_content"` / `android:layout_height="wrap_content"`: Sets the width and height to be just enough to contain the image.

* `android:scaleType="fitCenter"`: Adjusts the image to fit the center of its container proportionally.

* **Loading Images (Code Snippet Example):**

  * Store `myphoto.png` in a `res/drawable` (or density-specific) folder.

  * Declare an `ImageView` in XML: `<ImageView android:id="@+id/my_image_view" android:layout_width="wrap_content" android:layout_height="wrap_content" />`.

  * In Java:

    ```java

    // Declaration

    ImageView myImageView;

    // Initialization in onCreate()

    myImageView = (ImageView) findViewById(R.id.my_image_view);

    // Loading the image

    myImageView.setImageResource(R.drawable.myphoto); // Or set via XML: android:src="@drawable/myphoto"

    ```

9. Data Adaptors

* **Adapter Object:** Acts as a **bridge between an `AdapterView` and the underlying data** for that view. It provides access to the data items and is responsible for creating a `View` for each item in the data set.

* **Types of Data Adapters:** `ArrayAdapter`, `ListAdapter`, `CursorAdapter`.

* **AdapterView:** A `ViewGroup` that accesses data through an Adapter object.

* **Common AdapterViews:** `ListView`, `GridView`, `Spinner`, `Photo Gallery`.

* **AdapterView Responsibilities:**

1. Filling the layout with data received via the Adapter object.
2. Handling user selections (performing an action when an item is selected).

* **ArrayAdapter:**
    * **Purpose:** Used to wrap a Java array or `java.util.List` instance.
    * **Parameters:** Takes `Context` (current activity instance), `Layout` (for organizing items, e.g., `android.R.layout.simple_list_item_1`), and the actual `data source` (array or list).

Steps to Use an Adapter Object:

1. **Define a View object** (e.g., `ListView`, `Spinner`) in an XML layout file.
2. **Declare a Reference Variable** for the AdapterView within the activity class (Java file) (e.g., `ListView myListView;`).
3. **Initialize Reference Variable** with the ID of the AdapterView by calling `findViewById()` within the `onCreate()` method (e.g., `myListView = findViewById(R.id.my_list_view);`).
4. **Define the Source of Data** within `onCreate()` (e.g., an `ArrayList` initialized with data items).
5. **Define an Adapter Object** within `onCreate()` (e.g., `ArrayAdapter<String> dataAdapter = new ArrayAdapter<>(this, android.R.layout.simple_list_item_1, myArrayList);`).
6. **Define the Layout Style** for organizing items on `AdapterView` (often done in step 5 as a parameter to the adapter).
7. **Register (Attach) the Data Adapter Object** on the AdapterView (e.g., `myListView.setAdapter(dataAdapter);`).
8. **Implement `OnItemSelectedListener`** for detecting items selected by the user, and define the event handlers (`onItemSelected`, `onNothingSelected`) that will be triggered.

10. Introduction to PHP

* The provided sources **do not contain any information regarding PHP**. Therefore, I cannot provide details on this topic based on the given material. (Information about PHP would need to come from external sources, which you may want to independently verify.)

11. Data Storage and Retrieval

* The Android system grants access to the **file system** as well as an **embedded SQLite database**.

* **Content Providers** are objects that can retrieve and store data in data storage mechanisms such as **SQLite** or data files. They are the only way to share data across packages/applications.

* **Additional Information (External to Sources):** Beyond SQLite and the general file system, Android provides several mechanisms for data storage:

    * **Shared Preferences:** For storing small collections of key-value pairs of primitive data.

    * **Internal Storage:** For saving private data on the device filesystem that only your app can access.

    * **External Storage:** For saving public data on shared external storage (e.g., SD card).

    * **Network Storage:** For saving data on a web service or cloud database via network APIs.